Informe 1.a: Desarrollo de la ontología del dominio

1. Decisiones principales de diseño y jerarquía

El diseño de la ontología se ha centrado en establecer los conceptos, propiedades y relaciones fundamentales para representar los mitos de Andrómeda y el Laberinto de Creta, delegando la lógica de ejecución al sistema inteligente (Drools).

- Dejamos *Personaje* y *Criatura* como clases abstractas porque en nuestro proyecto siempre crearemos tipos concretos (Dios, Héroe, Monstruo, etc.).

- La clase Objeto la hemos puesto concreta para poder meter materiales básicos como la cera o las plumas. Para los demás objetos, hemos creado subclases más específicas como *Arma, ObjetoMagico* o *Trofeo* (para poder guardar las cabezas de los monstruos que derrotemos).

- Para los personajes que actúan juntos (como las Ninfas o las Grayas), en lugar de crear un personaje por cada una, hemos creado la clase *Grupo*.

2. Modelado de Héroes mediante Herencia Múltiple

Para solucionar que un héroe pueda ser un semidiós o un humano normal, le daremos a los personajes en Protégé las dos clases a la vez (herencia múltiple). Por ejemplo, Perseo pertenecerá a la clase *Heroe* y a la clase *Semidios*, mientras que Teseo será *Heroe* y *Humano.* Así luego será más fácil programar las reglas generales para todos los héroes.

3. Atributos Dinámicos y Estados (Data Properties)

Para saber la situación de un personaje o un objeto, hemos decidido usar atributos (slots) de tipo booleano (verdadero/falso) como estaVivo, estaLibre y estaRoto. Es la forma más sencilla de que luego Drools pueda cambiar estos datos rápidamente durante la ejecución (por ejemplo, si un objeto se rompe).

4. Desacoplamiento de Capacidades y Objetos Mágicos

Las capacidades (como volar o ser invisible) no se han modelado como variables booleanas estáticas en los personajes. En su lugar, se hemos creado una clase raíz *Capacidad* con sus propias subclases (*Vuelo, Invisibilidad, Petrificacion, Reflejo, Fuerza*).

Los objetos y animales mitológicos se relacionan a estas capacidades a través de la propiedad *otorgaCapacidad* (por ejemplo, las SandaliasAladas otorgan Vuelo). De esta forma, enlazamos los objetos con la capacidad que otorgan. Esto nos servirá luego en Drools para que el héroe pueda volar si tiene *cualquier* objeto que dé vuelo (sean las sandalias, el conjuro o las alas), dándonos mucha flexibilidad para el desarrollo de la narrativa.

5. Decisiones Narrativas y Simplificación Causal

Para garantizar que las cadenas causales se ejecuten correctamente a partir de los hechos iniciales, se han tomado las siguientes decisiones sobre la interpretación de los mitos:

- **Enojo divino:** Se ha establecido a Poseidón como el ofendido por Casiopea, descartando a Doris para minimizar el número de pasos lógicos en la liberación de Ceto.
- **Obtención de vuelo:** Las SandaliasAladas son entregadas a Perseo por las Ninfas. De esta manera aseguramos que Pegaso solo aparezca cuando muera Medusa, respetando esa versión del mito.

6. Separación entre Protégé y Drools

Aunque en Protégé hemos creado clases como Accion y Mision, hemos tomado la decisión de que la ontología funcione únicamente como el "esqueleto" o el mapa estático de nuestro proyecto. Es decir, Protégé solo se encarga de definir qué elementos pueden existir en el mundo (quién puede ser un dios, qué tipos de armas hay o qué capacidades existen).

Toda la historia, es decir el encadenamiento de los sucesos, las decisiones, los combates y los cambios de estado (como que un personaje muera, sea apresado o un objeto cambie de dueño), ocurrirá exclusivamente en el motor de reglas de Drools. Las instancias que estamos introduciendo ahora mismo en Protégé (como crear al individuo "Perseo" o poner que "Hermes" tiene la "Hoz de Acero") son meramente instancias de prueba. Sirven solo para comprobar que nuestro diseño de carpetas y propiedades funciona bien. En la versión final del programa, será Drools quien lea los archivos de texto de cada escenario y vaya creando o modificando los personajes y objetos "al vuelo" según lo que pida la historia.

7. Uso de Inteligencia Artificial Generativa

En esta primera fase, hemos usado la inteligencia artificial Gemini únicamente como apoyo para entender e interpretar el enunciado al principio. Le pedimos a la herramienta que analizara los mitos para ver un poco más claros los requisitos

de la historia y entender qué personajes, objetos y relaciones íbamos a necesitar. Una vez tuvimos esa idea inicial clara, todo el trabajo de análisis, el diseño de las clases y la creación de la ontología en el programa Protégé lo hemos hecho nosotros mismos por nuestra cuenta.